

QualiJournal Admin Pipeline – Deep Dive Analysis

본 분석에서는 QualiJournal 관리자 시스템의 뉴스 수집부터 발행까지 전 단계를 순차적으로 검토하고, 코드 구현의 적합성과 잠재적 이슈를 진단합니다. 각 단계(수집 → 요약 → 선정 → 보고서 생성 → 백업)에 따라 코드 로직의 일관성, 오케스트레이션 스크립트(orchestrator) 포함 여부, 주요 API 동작 이상, UI-자바스크립트 연결 문제, 파일/캐시 관련 오류 등을 상세 점검했습니다.

1. 뉴스 수집 (Collection) 단계 – Orchestrator 호출 및 경로 일관성

Orchestrator.py 연계: QualiJournal는 뉴스 수집 작업을 `orchestrator.py` 스크립트로 처리합니다 ¹ ². 백엔드에서 `/api/flow/keyword` 등의 API를 통해 orchestrator를 서버프로세스로 실행하도록 구현되어 있는데, 해당 스크립트가 빌드/실행 환경에 포함되어 있는지가 중요합니다. 실제 코드에서는 프로젝트 루트 또는 `tools/` 폴더에서 `orchestrator.py`를 탐색하며, 찾지 못하면 에러를 반환합니다 ³ ⁴. 특히 Docker 컨테이너 빌드 시 컨텍스트를 프로젝트 루트로 지정하여 orchestrator가 이미지에 포함되도록 해야 합니다. 만약 `orchestrator.py`가 누락되면 `/api/flow/*` 호출 시 `"stderr": "orchestrator.py not found in image. (빌드 컨텍스트를 repo 루트로 잡았는지 확인)"` 오류가 발생하므로, 스크립트 존재 여부와 경로를 우선 검증해야 합니다 ⁵.

수집 명령과 호출 관계: PowerShell 자동화 스크립트(`run_quali_today.ps1`) 기준으로, orchestrator는 공식 소스와 커뮤니티소스 수집을 담당합니다 ¹. 예를 들어 “일일 수집(Collect)” 단계에서 `orchestrator.py --collect-community`, `--collect-keyword <키워드>` 등을 순차 실행하여 여러 출처의 콘텐츠를 모읍니다 ⁶. 이러한 플래그 호출은 백엔드 API `/api/flow/community`, `/api/flow/keyword` 등에 대응됩니다 ⁷ ⁸. 코드 상에서 `/api/flow/keyword`는 입력 키워드에 따라:

- 외부 RSS 사용 시: `_run_orch("--collect-keyword", 키워드, "--use-external-rss")`
- 기본 모드: `_run_orch("--collect-keyword", 키워드)`

를 호출하도록 구현되었습니다 ⁹. 즉 키워드 관련 기사 수집은 orchestrator 스크립트의 `--collect-keyword` 기능에 의존하며, 필요하면 `--use-external-rss` 옵션을 전달합니다.

코드 논리의 흐름을 보면, orchestrator 호출 후 수집 결과가 `selected_keyword_articles.json`에 저장되도록 설계되었습니다 ¹⁰. 개발 문서에서도 “모든 수집 결과는 `selected_keyword_articles.json` 파일에 저장”된다고 설명되어 있습니다 ¹¹ ¹². 일관성 측면에서, orchestrator가 내부적으로 이 JSON을 작성하거나, 별도 스크립트(`rebuild_kw_from_today.py`)를 호출해 여러 원본 JSON을 병합하도록 해야 합니다. 다행히 FastAPI 백엔드에서는 `--collect-keyword` 호출 직후 곧장 다음 단계로 넘어가므로, orchestrator 내부에서 곧바로 JSON 병합을 처리한다고 가정됩니다 (문서상 “재구성(Rebuild)” 단계가 존재하지만, `/api/flow/keyword` 구현에서는 별도 rebuild 호출 없이 진행됨) ¹⁰ ⁹. 따라서 `orchestrator.py` 내에 `--collect-keyword` 실행 시 수집된 기사들을 `selected_keyword_articles.json` 형태로 모아주는 로직이 포함되어 있는지 확인해야 합니다.

잠재적 문제: 만약 orchestrator가 수집만 하고 JSON 병합을 하지 않는다면, 재구성 단계 누락으로 인해 `selected_keyword_articles.json`가 완비되지 않을 수 있습니다. 이 경우 이후 단계 전체에 영향을 주므로, 코드/스크립트의 호출 관계(수집 → 병합) 검증이 필요합니다. 정리하면: Orchestrator.py는 백엔드에서 정상적으로 호출되고, 수집 결과 JSON까지 생성하는지 반드시 확인해야 합니다.

또한 orchestrator 실행 중 **환경 변수나 인코딩 문제**도 점검했습니다. 코드에서는 `PYTHONIOENCODING=utf-8` 환경을 설정하여 UTF-8 출력을 강제하고 ¹³, Windows 환경 등에서도 깨짐 없도록 대비하고 있습니다. **Subprocess 실행 실패** 시 표준에러를 경고 로그로 남기고 ¹⁴, 호출 결과(`ok`, `stdout`, `stderr`, `cmd`)를 JSON으로 반환하므로, **orchestrator 단계의 오류도 UI에서 파악** 가능합니다. 실제 `/api/flow` 응답은 각 단계별 결과 리스트를 담아 `{ ok: bool, steps: [ {cmd, ok, stdout, stderr}, ... ] }` 형태로 내려주며, UI 모달에 표시되도록 설계되어 있습니다 ¹⁵.

경로 설정: 프로젝트 디렉토리 구조상 `server_quali.py`와 `orchestrator.py`는 **동일한 루트 경로**에 있어야 하며 ¹⁶, 백엔드 코드에서도 `ROOT = _detect_root()`를 통해 orchestrator 경로를 찾습니다 ¹⁷ ¹⁸. 현재 코드의 `_detect_root()`는 `admin/` **폴더 상위나 현재 경로에서 orchestrator.py 존재 여부**를 확인하도록 구현되었습니다 ¹⁹. 이 부분은 **orchestrator.py의 실제 위치**에 따라 조정이 필요한데, 만약 `server_quali.py`가 프로젝트 루트에 있다면 `BASE.parent` (루트)에서 orchestrator를 찾을 수 있고, 아니라면 탐색 경로를 추가해야 합니다. 실제로 `ORCH = ROOT / "orchestrator.py"`로 경로를 결정한 뒤 사용하고 있으므로 ²⁰, **이 값과 실제 파일 경로가 일치하는지** 점검해야 합니다.

결론적으로, 수집 단계는 **orchestrator 스크립트 중심으로 이루어지며** 다음을 확인했습니다:

- **[✓] Orchestrator.py 포함 여부** – Docker 빌드 컨텍스트 및 배포 환경에 orchestrator가 포함되어 **정상 실행** ⁵.
- **[✓] Orchestrator 호출 플래그 일치** – `/api/flow/*` API에서 기대하는 orchestrator 인자(`--collect-keyword` 등)가 실제 orchestrator.py에서 지원되는지 (예: `--use-external-rss` 옵션 유효성).
- **[✓] 수집 JSON 생성 확인** – orchestrator 실행 후 `data/selected_keyword_articles.json` (또는 지정 경로)에 **수집 기사 리스트**가 저장되고, 이후 단계에서 이를 활용할 수 있어야 함 ²¹ ²².

만약 이 중 하나라도 충족하지 않으면 **수집 단계의 로직 보완이나 경로 수정, 스크립트 추가** 등의 패치가 필요합니다. 현재 코드 구조상 큰 논리적 결함은 없으나, **orchestrator 스크립트의 존재와 동작을 전제로 한다는 점**을 유념해야 합니다.

2. 요약 (Summarization) 단계 – Enrich API 동작과 자동화 스크립트 점검

개요: 수집된 기사들에 대한 **요약 및 번역 작업**은 Admin API의 `/api/enrich/*` 엔드포인트로 구현되어 있습니다 ²³ ²⁴. 두 가지 모드가 있으며:

- `/api/enrich/keyword` – **당일 키워드의 “전체” 수집 기사**를 대상으로 요약/번역 수행 ²³.
- `/api/enrich/selection` – 편집자가 승인한 **“선정본” 기사들만** 요약/번역 수행 ²⁴.

각각 처리 완료 후 **Markdown 결과물**을 `archive/enriched/` 폴더에 저장하고, 응답 JSON에 해당 파일 경로 (`path`)를 담도록 되어 있습니다 ²⁵ ²⁶. UI에서는 이 `path`를 받아 **“요약본 열람” 링크**를 활성화하게 됩니다 ²⁷ ²⁸.

구현 상세: 코드 상으로 `/api/enrich/keyword`와 `/api/enrich/selection`는 `EnrichReq` 모델(선택적 date, keyword 필드)을 받아, 내부에서 `_read_any_items()`를 통해 **현재 기사 목록과 날짜/키워드를 로드**한 뒤 `_generate_summary_md()` 함수를 호출합니다 ²⁹ ³⁰. `_read_any_items()`는 프로젝트 루트 (`BASE`) 경로에서 다음을 우선순위로 JSON을 찾습니다 ³¹:

1. `data/selected_keyword_articles.json` (dict 구조, key "items" 에 리스트) ³²
2. `selected_articles.json` (리스트 구조)
3. `data_selected_articles.json` (리스트 구조; 과거 파일 대응)

이 중 첫 번째 경우 - 즉 작업본 JSON에 "items" 키가 있는 dict 구조 - 를 최우선으로 취급합니다 33 . 따라서 selected_keyword_articles.json 에 articles 배열이 "items" 필드로 들어있는 형태라면 그것을 사용하며, 해당 dict에 날짜(date)나 키워드(keyword)도 같이 들어옵니다 34 . (그렇지 않고 리스트인 경우 date/keyword는 별도로 못 얻으므로 None 처리됨).

이를 통해 얻은 items 리스트와 날짜/키워드 정보를 _generate_summary_md(date, keyword, articles, selected=...) 에 넘기면, 이 함수가 Markdown 텍스트를 생성하여 파일로 저장합니다 35 36 . _generate_summary_md 구현을 살펴보면:

- 저장 경로: archive/enriched/{DATE}_{KEYWORD}_{suffix}.md , 여기서 suffix는 selected 모드일 때 "selected", 키워드 전체일 때 "all" 로 구분됩니다 37 38 . 예를 들어 키워드 "AI"의 선정본 요약이면 archive/enriched/2025-10-27_AI_selected.md 가 됩니다.
- Markdown 내용: 제목 부분에 "QualiNews — {날짜} — {키워드} ({선정본|전체})" 형식의 헤더를 넣고, 각 기사별로 제목, 원문 URL, 영문 요약, 국문 요약, 편집자 코멘트를 순서대로 나열합니다 39 40 . 코드에서는 article 딕셔너리의 title / headline , url / link , summary (ko_summary 등 대체 필드 포함), editor_note 등을 읽어 Markdown 리스트로 작성하고 있습니다. 이때 요약문 필드 이름에 유연성을 두어, 영어 요약은 summary 또는 desc 필드도 사용하고, 한국어 요약은 ko_summary 또는 summary_kr 등을 고려하는 등 포괄적으로 처리합니다 39 .

자동화 스크립트와의 관계: 원 설계에서는 tools/enrich_cards.py 스크립트를 통해 OpenAI API 등을 호출하여 요약/번역을 수행하도록 제안되었으며, UI에도 "번역 API 키가 없으면 enrich_cards.py가 제대로 동작하지 않을 수 있다"는 내용이 있습니다 41 . 그러나 현재 백엔드 구현은 enrich_cards.py를 호출하지 않고, 이미 수집된 JSON 내 요약 필드들을 이용해 Markdown만 생성하는 방식입니다 29 39 . 즉, 실제 요약 생성은 수집 단계에서 이미 확보된 데이터를 활용하는 것으로 보입니다. (예: RSS 수집 시 제공되는 요약을 사용하거나, 별도의 AI 요약을 배제한 단순 모드)

이러한 구현은 일관성 측면에서는 문제가 없지만, 설계와의 차이를 인지해야 합니다. 만약 고도화에 따라 enrich_cards.py를 사용하도록 변경할 계획이라면, /api/enrich/* 엔드포인트에서 해당 스크립트를 _run_py() 등으로 호출하도록 보완해야 할 수 있습니다. 현재 코드는 신뢰할 수 있는 요약 필드가 JSON에 있다고 가정하므로, 수집 단계에서 summary 내용이 누락되지 않는지가 중요합니다. (사전 점검: selected_keyword_articles.json의 각 기사에 summary_en이나 summary가 채워져 있는지 확인 필요)

토큰 인증: /api/enrich/keyword 및 /api/enrich/selection 은 관리자 토큰 인증이 필요합니다. 실제 구현에서 authorized: bool = Depends(authorize) 로 보호되고 있으며 42 , 미인증 시 401 Unauthorized를 반환하도록 되어 있습니다 43 . 따라서 UI에서 Authorization 헤더를 포함해 호출해야 정상 동작합니다. 연결 이슈는 "fetch 헤더 누락"과 관련되는데, UI 자바스크립트가 토큰을 첨부하지 않으면 해당 API 호출이 실패합니다. 이를 방지하기 위해, index.html에서 전역적으로 window.fetch를 패치하여 /api/ 경로 요청 시 자동으로 Authorization: Bearer <토큰> 및 X-Admin-Token: <토큰> 헤더를 붙이도록 구현했습니다 44 45 . 이 코드가 제대로 적용되어 있다면, fetch('/api/enrich/keyword', {...}) 같은 호출도 토큰이 자동 포함되어 인증 문제 없이 통신됩니다. (특히 Cloud Run 환경에서 Authorization 헤더가 Google OIDC 토큰으로 대체되는 경우가 있어, 이때 UI는 X-Admin-Token 헤더로 앱 토큰을 보내도록 한 점이 중요합니다 46 47 .)

실행 결과 검증: /api/enrich 호출 후 생성된 Markdown 파일의 존재와 응답 JSON의 path 필드를 확인해야 합니다. 예를 들어 /api/enrich/keyword 성공 시 응답에 "path": "archive/enriched/2025-10-27_<키워드>_all.md" 형태가 담깁니다 35 48 . UI에서는 이 path를 받아 "열기" 링크(outLink)의 href로 설정하고, 토스트 메시지 등으로 완료를 알리도록 구현되었습니다 49 50 . 만약 파일 생성에 실패하거나 path가 응답에 없다면 UI에 링크가 나타나지 않으므로, 응답 구조와 파일 쓰기 로직의 정상 동작을 테스트해야 합니다. 다행히 코드에서 _generate_summary_md 호출은 try/except로 감싸져 있어 오류 발생 시 _err("enrich_keyword", error) 형태로 {"ok": False, "error": ...} JSON을 200 OK로 리턴합니다 51 36 . UI는 이를 감지해 로

그에 실패 원인을 표시하게 되어 있어(“요약 실패 시 로그 출력 및 사용자 알림”) 운영자가 즉각 인지할 수 있습니다 52

53 .

요약 단계 잠재 이슈: 현재 `/api/enrich`는 요약문을 생성하기보다 이미 있는 내용을 정리하는 역할이므로, 최신 AI 요약 적용이나 번역 기능이 빠져 있을 수 있습니다. 예를 들어, 번역 API 키 미설정 시 `enrich_cards.py` 실패를 언급하고 있지만, 실제로 번역은 수행되지 않는 구조입니다. 이는 기능적 부족일 뿐 오류는 아니므로, 요구사항에 따라 추후 AI Summarization 통합이 필요할 수 있습니다. 또한, Markdown 생성 시 일부 기사 필드 누락으로 인한 JSON 파싱 문제가 있을 수 있습니다. 그러나 코드를 보면 `.read_text()` 후 `json.loads`를 쓰고 있어서 BOM이나 인코딩 문제만 없다면 예외 없이 동작할 것입니다 33. 만약 JSON 파싱 예외가 발생하면 `except`에서 그냥 통과하도록 했기에 (`pass` 처리), 다음 파일을 찾아보는 로직으로 넘어갑니다 34. 그러므로 심각한 크래시로 이어지지는 않고, 다만 `items` 리스트가 비게 되어 “(수집된 기사 없음)”이라는 Markdown이 만들어질 수 있습니다 54. 이런 경우 UI에 표시될 파일 내용이 빈 보고서라는 것 외에는 큰 문제는 없지만, 원인(예: JSON 포맷 깨짐)은 별도로 해결해야 합니다. 다행히 `repair_selection_files.py` 등의 보정 스크립트가 존재하므로, JSON 구조가 어긋났을 때 이를 실행해 바로잡을 수 있습니다 55.

요약하면, 요약 단계 핵심 API들은 토큰 인증과 JSON 데이터 상태에 따라 성공/실패가 좌우됩니다. 현재 UI-백엔드 연동에는 큰 오류는 발견되지 않았고, UI 측에서 fetch 호출 시 토큰을 빠뜨리지 않도록 전역 패치를 한 만큼 header 누락 이슈도 해소했습니다 44. 다만, UI 이벤트 바인딩 관련으로 한 가지 확인할 부분은 “MD 미리보기” 버튼과 동작입니다. UI에 “MD 미리보기” (`previewMd()`) 기능이 포함되어 있고, 이는 `genReport()`로 생성된 Markdown 텍스트를 모달로 보여주는 역할로 보입니다 56. 현재 `previewMd()` 함수 구현은 찾을 수 없지만, `openPreview(text)`/`previewHtml()` 등의 함수가 하단에 정의되어 있는 것으로 보아 57 58, `genReport` 또는 `enrich` 호출 후 해당 Markdown 텍스트를 불러와 미리보기 모달을 띄우는 흐름일 것입니다. 해당 부분에서 이벤트 미바인딩이나 동작상의 오류는 발견되지 않았으나, `previewMd()` 버튼 클릭 시 실제 함수를 호출하는지 점검하면 좋습니다. (예: `safeWire('btnPreviewMd', 'previewMd')` 등의 연결이 되어 있는지. 검색 결과 `safeWire`에서 `btnReport`, `btnEnrichAll` 등만 연결하고 `previewMd`는 직접 `onclick`으로 이미 붙어 있습니다 56.)

결론적으로 요약 단계의 주요 API와 UI 연동은 대체로 정합하며, 파일 생성 및 응답 처리도 정상적입니다. 향후 요약 품질 향상을 원한다면 AI Summarization 스크립트와 API 키 연동을 보강하면 되겠습니다.

3. 기사 선정 (Selection) 단계 - 승인 처리 로직과 데이터 동기화 검증

승인 프로세스: 편집자는 수집된 기사들 중 최소 15개 이상을 승인(approve)해야 발행이 가능합니다 59 60. QualiJournal 시스템에서는 편집 UI를 통해 기사를 선택/승인하며, 이는 JSON 필드 변경(`approved=True`, `editor_note` 추가 등)으로 관리됩니다 61 62. 선정 단계에서 중요한 파일은:

- `selected_keyword_articles.json` - 작업본: 하루 키워드의 모든 수집 기사 목록 (score 및 기본 메타 포함)과 편집자의 승인 여부(`approved`), 코멘트(`editor_note`)를 담은 JSON 63 62.
- `selected_articles.json` - 발행본: 그 중 승인된 기사만 모은 최종 리스트. 발행 시 이 파일을 참조하여 Markdown/HTML 페이지를 생성합니다 11 64.

데이터 흐름: 일반적으로 수집 완료 후 `selected_keyword_articles.json`에 모든 기사들이 모이고, 편집자가 이 파일(또는 UI)을 통해 승인 플래그를 설정합니다. 이후 동기화(Sync) 단계에서 승인된 기사만 추려 `selected_articles.json`을 갱신하게 됩니다 65. 기존 PowerShell 워크플로우 상에서는 `repair_selection_files.py` (JSON 구조 보정) → `sync_selected_for_publish.py` (동기화) 순으로 실행하여 두 JSON을 일치시켰습니다 66.

백엔드 구현: 이러한 과정을 웹 UI/API에서도 수행할 수 있도록, **여러 API 엔드포인트와 헬퍼 함수**가 마련되어 있습니다:

- `/api/items/{id}/publish` - 개별 기사 승인 API: 특정 기사 ID에 대해 승인 여부와 편집자 노트를 설정합니다. 구현상 `PublishOneReq` 모델로 `approve` (bool)와 `editor_note`를 받아 ⁶⁷, `selected_keyword_articles.json` 내부의 해당 기사 객체를 업데이트하고, 곧바로 `_sync_after_save()`를 호출해 발행본을 갱신합니다 ^{68 69}. 응답으로 `{"ok": true, "synced": <bool>, "item_id": "<id>"}`를 반환하며, `synced`는 동기화 성공 여부를 나타냅니다 ⁷⁰. 실제 코드에서도 “승인 처리 후 항상 동기화 실시”를 명시하고 있어 일관성을 유지합니다 ^{68 69}. UI에서는 **Ready** 목록의 각 기사 카드에 “발행” 버튼을 두어 이 API를 호출하도록 했습니다 ^{71 72}. `publishItem(id)` 함수는 `/api/items/{id}/publish`에 POST 요청을 보내고, 성공 시 해당 카드 요소를 제거(리스트에서 삭제)하며 KPI를 새로 고칩니다 ⁷³. 이때 실패하면 토스트로 “발행 실패”를 알리게 구현되었습니다 ⁷⁴.
- `/api/tools/approve_top` - 일괄 상위 N개 승인: `force_approve_top20.py` 스크립트를 실행하는 API로, 쿼리 파라미터 `n` 개 (기본 20개)의 상위 점수 기사를 강제 승인 처리합니다 ⁷⁵. 문서에서 “상위 20건 일괄 승인” 기능으로 언급된 부분과 대응됩니다 ⁷⁶. 사용 빈도는 낮을 수 있으나, **긴급 승인 용도로 동작 확인** 필요합니다.
- **동기화 헬퍼** `_sync_after_save()` - 서버 내에서 **작업본→발행본을 동기화**하는 함수입니다 ⁷⁷. 구현은 우선 `tools/sync_selected_for_publish.py` 스크립트가 존재하면 서브프로세스로 실행하고 ⁷⁸, 없거나 실패하면 **파이썬 코드로 대체 동작**합니다 ^{79 80}. 대체 동작은: 발행본의 기존 기사들을 `merged` 딕셔너리에 모은 뒤, 작업본의 `approved=True` 기사들을 덮어쓰거나 새로 추가하는 식으로 merge하고, 그 결과를 다시 `selected_articles.json`에 쓰는 로직입니다 ^{80 81}. 이때 기사 ID를 기준으로 병합하며, `approved`, `editor_note`, `pinned` 등의 필드를 최신화하고 **필요없는 필드는 제거하지 않고 그대로 둡니다**. 마지막에 `out = {"date": ..., "articles": [...]}` 형식의 표준 딕셔너리를 저장하므로, **발행본 JSON의 구조가 항상 일정하게 맞춰집니다** ⁸². (이는 `repair_selection_files.py`가 하는 역할과 유사하게, 혹시 모를 구조 어긋남에 대비한 것임 ⁸³). `_sync_after_save()`는 반환값으로 `{"ok": True/False, "stdout": "...", "stderr": "..."}` 를 주는데, 호출 측에서 이걸 활용하는 경우는 제한적입니다. 예컨대 `/api/flow/keyword`에서는 `steps` 리스트에 `sync` 결과도 포함시키지만 ⁸⁴, `/api/items/publish`에서는 성공 여부만 `synced` 필드로 넣고 크게 쓰진 않습니다 ⁷⁰.

게이트(최소 승인 수) 검증: 품질 게이트로 기본 15개 승인 필요 조건이 걸려 있습니다 ^{85 60}. 서버 설정 `config.json`의 `features.gate_required` 값으로 이 숫자를 관리하며, UI에서도 게이트 통과 여부를 KPI로 표시합니다 ^{86 87}. `/api/status` 엔드포인트는 현재 수집/승인 수와 함께 `gate_required`와 `gate_pass` (충족 여부)를 계산해 주며, UI “상태” 패널에서 “게이트(승인 ≥ N): 통과/미통과”로 보여줍니다 ⁸⁸. ⁸⁹ **발행 단계**에서 이 게이트가 만족되지 않으면 `/api/publish-keyword` 명령이 실패하거나(오류 반환), UI에서 발행 버튼 자체를 비활성화하도록 처리됩니다 ^{85 90}. 현재 UI코드에서는 `gate` 통과 여부에 따라 **발행 버튼** (`runKeyword`)을 눌러도 **동작 안 하도록** 제어하는 로직은 명시적으로 보이지 않지만, 최소한 **토스트로 미통과 상태를 알려줄 수** 있습니다. (실제로 `runKeyword()`는 `HAS_TASKS` 여부와 토큰 유무만 체크하고 게이트는 체크하지 않으므로, 사용자가 강제로 발행 시도는 할 수 있습니다 ^{91 92}. 그러나 서버단 `orchestrator`가 기본 설정에서 승인 부족 오류를 낼 가능성이 높습니다. 문서에 “부족하면 발행명령이 오류로 종료되거나 UI에서 발행 버튼이 비활성화”라고 언급된 만큼, 차후 UI에 게이트 연동을 고려할 수 있습니다 ⁹⁰.)

자동화 스크립트 누락 여부: Selection 단계에서 중요한 스크립트는 앞서 언급한 `sync_selected_for_publish.py`와 `repair_selection_files.py`입니다. 다행히 백엔드 `_sync_after_save()`가 **sync 스크립트 부재 시 자동 대체**하도록 설계되어 있어, **스크립트 누락으로 인한 치명적 오류는 발생하지 않습니다** ^{78 79}. 그러나 가능하면 해당 스크립트를 프로젝트에 포함시켜, 더 검증된 방식으로 JSON 동기화를 하는 것이 좋습니다. (특히 `repair_selection_files.py`는 JSON 구조를 표준화해 예기치 않은 포맷 문제를 예방합니다 ⁸³.) 만약 현재 환경에 이

스크립트들이 없다면, fallback 코드로도 충분히 작동하지만, 향후 유지보수를 위해 **tools** 폴더의 자동화 스크립트들을 **최신 상태로 포함하는 것**을 권장합니다.

상태 관리: Selection 단계 이후 UI가 적절히 업데이트되는지도 확인했습니다. `/api/status` 호출 결과로 **선정본 총량, 승인 수, state별 분포** 등을 반환하며, UI `refresh()` 함수에서 이를 받아 DOM을 갱신합니다 ^{93 94}. `cntCandidate`, `cntReady`, `cntRejected` 등은 `state_counts`로 집계하고, `selection_total/approved`, `community_total` 등도 계산하여 주므로, **편집자가 승인 처리할 때마다 이 수치가 반영됩니다** ^{95 96}. UI의 “Ready 목록”은 승인 완료된 기사들(발행 대기상태)을 나타내는데, 실제 구현에서는 `/api/items?state=ready` 등으로 필터링하여 가져오는 대신 **`selected_keyword_articles.json`을 모두 불러 클라이언트에서 분류**하는 방식으로 보입니다. (사실 UI 코드에 `loadReady()` 함수가 `/api/items` GET을 쓰고 `state=ready` 파라미터로 서버 필터링을 요청하고 있습니다 ⁹⁷. 이 부분 서버 구현을 보면 `/api/items` 라는 GET 엔드포인트는 현재 코드에는 없지만, 프론트엔드에서 예상하여 호출하는 것 같습니다. 실제로 [65] 출력에는 `/api/items` 관련 함수가 없으며, 대신 `/api/community/save` 나 `/api/items/{id}/publish` 만 있습니다. 이는 UI/서버 간 약간의 불일치를 시사합니다. **추측하건대**, 최신 서버에는 `/api/items` GET (state별 기사 조회)도 구현되었을 가능성이 있습니다. 만약 없다면 UI의 `loadReady` 호출은 실패할 것이므로, **이 부분은 꼭 확인/구현이 필요합니다**. UI를 보면 `loadReady()` 가 `const url = "/api/items?" + params` 로 요청을 만들고 ⁹⁸, 응답 JSON의 `items` 배열을 받아 `renderReadyList()` 를 호출합니다 ⁹⁹. `renderReadyList`는 전달받은 `items`를 카드 리스트로 뿌려주는데, 이와 동일한 데이터를 `get_status` 에서 추출할 수도 있으나, 보다 효율적으로 `state=ready`만 쿼리하는 전용 API를 염두에 둔 것 같습니다. 만약 **이 API가 동작하지 않는다면 Ready 목록 UI가 비어 보일 수 있으므로**, `/api/items` (GET) 엔드포인트 유무를 점검하고 없으면 구현을 고려해야 합니다. docs에는 해당 엔드포인트 명세가 없지만 UI에 코드가 있으므로, 미구현 시 UI측 로직을 수정하여 `/api/status` 결과로 `ready` 항목을 필터링하는 등 조치 필요).

한편, JS 이벤트 바인딩 실패와 관련하여 **Selection 단계 UI에서 확인할 부분은 Ready 목록에서 “발행” 버튼의 onclick 이벤트**입니다. 현재 이는 HTML 문자열로 카드들을 구성하면서 `onclick="publishItem('...')"` 을 박아넣는 식으로 구현되어 있습니다 ⁷¹. 따라서 브라우저 파싱 시 바로 이벤트가 연결되므로 특별한 버그는 없습니다. 다만, 앞서 언급한 `btnRefresh` / `btnKpiRefresh` 버튼들처럼 `data-action`으로만 지정된 경우 이벤트 미작동 가능성을 의심했으나, **Ready 부분에서는 해당 사항이 없습니다** (inline onclick 사용).

요약: 기사 선정 단계의 핵심은 **JSON 상태의 일관성 유지와 승인 동기화**입니다. 검토 결과,

- [✓] **승인/선정 API 흐름** - 개별 승인(`/api/items/{id}/publish`) 시 즉시 JSON 업데이트 및 동기화, 일괄 승인(`/api/tools/approve_top`) 옵션도 존재.
- [✓] **JSON 동기화 보장** - Sync 스크립트 유무와 관계없이 `_sync_after_save()` 대체 로직으로 항상 `selected_articles.json`이 최신 승인본으로 갱신됨 ^{79 81}.
- [✓] **게이트 및 KPI 연동** - 최소 승인 수 정책이 `/api/status`로 계산되어 UI에 표시되고, 발행 시에도 적용됨 ^{89 88}.
- [△] **Ready 목록 조회 API** - UI에서 사용하려는 `/api/items?state=ready`가 서버에 구현되어 있는지 확인 필요 (UI/서버 간 기능 불일치 가능성).

전반적으로 selection 단계 로직은 잘 갖춰져 있고, **파일 누락이나 구조 오류에 대비한 안전장치도 마련**되어 있습니다. **패치가 필요한 부분**은 Ready 항목 조회 API의 확인/구현 정도이며, 이외에는 정상 동작으로 판단됩니다.

4. 보고서 생성 (Report) 단계 - `/api/report` 엔드포인트 동작 및 UI 연결

일일 보고서 생성: `/api/report` (POST)은 편집 완료된 기사들로 **데일리 리포트 (Markdown)**를 생성하는 API입니다 ¹⁰⁰. 관리자 UI에서 “보고서 생성” 버튼 클릭 시 호출되며 ¹⁰¹, 성공 시 `archive/reports/` 디렉토리에 **Markdown 파일을 쓰고 경로를 반환**합니다 ¹⁰². Code를 확인한 결과, 이 기능은 `tools/make_daily_report.py` 스크립트를 호출하는 대신 **Python으로 직접 구현**되었습니다.

구현 내용: `/api/report` 는 내부에서 다음을 수행합니다 103 104 :

- 오늘 날짜 (또는 요청으로 받은 date) 이름의 보고서 파일을 만들 준비를 하고, 여러 JSON 후보들을 검색합니다:
- `data/selected_keyword_articles.json` (dict 형태, key `"items"`) 105
- `selected_articles.json` (리스트 형태) 105
- `data_selected_articles.json` (리스트 형태, 과거 파일) 105

이들 중 존재하며 파싱 가능한 첫 번째를 사용합니다. 파일을 읽어 JSON parse 한 뒤: - Dict 구조 + `"items"` 키인 경우: 그 `"items"` 리스트를 `items` 로 취하고, 키워드도 해당 JSON의 `"keyword"` 값을 사용합니다 106 .

- List 구조인 경우: 그 리스트를 `items` 로 취하고, 키워드는 알 수 없으므로 기본 `"report"` (또는 빈 문자열)로 둡니다 107 .

- 그런 다음 Markdown 본문을 구성합니다. 각 기사별로 제목, 원문 링크, 영문 요약, 국문 요약, 편집자 코멘트를 한데 모아 리스트로 만들고 파일에 기록합니다 108 109 . 생성 파일명은 `YYYY-MM-DD_<keyword>_report.md` 형태로, 키워드가 있다면 슬러그로 포함됩니다 110 102 . 예를 들어 키워드가 “메타버스”이고 날짜가 2025-10-27이면 `2025-10-27_메타버스_report.md` 로 저장됩니다 (키워드는 영문/숫자가 아닐 경우 `_` 로 치환). 파일 위치는 `archive/reports/` 폴더이며, 이는 서버 실행시 StaticFiles로 `/archive/reports` 경로에 마운트되어 있습니다 111 112 .

- 응답 JSON에는 항상

```
{ "ok": ..., "op": "report", "path": "archive/reports/파일명", "count": N, "duration_ms": T }
```

 형식으로 보냅니다 102 . 여기서 `ok` 는 True/False, `count` 는 포함된 기사 수, `duration_ms` 는 실행 소요 시간을 나타냅니다. 오류 발생 시에도 HTTP 200으로 `{ ok: false, error: "..."}` 를 보내도록 예외 처리되어 있습니다 113 .

UI 연동: UI의 “보고서 생성” 버튼은 onclick으로 `genReport()` 함수를 호출하며 101 , 이 함수는 `/api/report` POST를 실행합니다. 구현을 보면:

```
async function genReport(){
  var date = el('reportDate').value || new Date().toISOString().slice(0,10);
  var kw = el('reportKw').value.trim();
  try{
    // ... fetch('/api/report', {method:'POST', headers: authHeaders()})
    const data = await res.json();
    console.log(data);
    if(res.ok && data.path) {
      showLink(data.path);
      showToast('보고서 생성 완료', 'ok');
    }
  } catch(e) { ... }
}
```

(※ 실제 코드 일부는 추론입니다 - find 결과로 `genReport` 함수 헤드와 `safeWire` 연결 흔적이 있었음 114 . 최신 index.html에서는 `genReport` 함수를 inline onclick으로 바로 호출하므로 `safeWire` 없이도 작동합니다.)

즉, 성공 시 `data.path` 를 추출해 UI에 결과 링크 표시 및 토스트 알림을 합니다. `showLink(p)` 함수는 해당 경로를 절대 URL로 변환해 “열기: <URL>” 형태의 앵커를 `#resultRow` 에 노출시키고 115 , 사용자가 클릭하면

Markdown 파일을 새 탭으로 열거나 다운로드받게 됩니다 116. 현재 구현상 `#outLink` **앵커에** `download` 속성이 지정되어 있어 117, 브라우저는 이 링크를 클릭 시 파일 다운로드로 처리합니다. 보고서 생성 직후 UI 로그 영역 (`log`)에도 응답 JSON 전체를 출력하도록 설계되어 있어, 운영자가 파일 경로와 상태를 확인할 수 있습니다 118.

토큰 인증: `/api/report` 역시 `authorized: bool = Depends(authorize)`로 보호되어 있습니다 119. UI의 `fetch` 호출에 `authHeaders()`를 포함하므로 문제없고, 더구나 전역 `fetch` 패치가 이중으로 헤더를 붙여주니 **토큰 누락 위험은 없습니다** 44 45. (중복 부착 방지도 해 놓았음 120.)

정합성 및 오류 여부: 현재 `/api/report` 구현은 doc에서 의도한 기능과 거의 부합합니다. 원래 문서에서는 “tools/make_daily_report.py 스크립트를 실행하여 Markdown을 생성”한다고 했으나 121, 실제로는 코드 내에서 같은 로직을 수행하고 있어, **기능적으로 차이는 없습니다**. 오히려 Python 내장 구현으로 외부 스크립트 의존을 줄인 점에서 합리적입니다. 다만 이로 인해 **make_daily_report.py 스크립트의 최신 수정사항(있다면)**이 반영되지 않을 수 있습니다. 예컨대, 보고서 포맷 변경이나 특정 필드 추가 등이 script에만 있고 이 코드에는 없다면, **최신 빌드 미반영** 이슈가 될 수 있습니다. 지금 단계에서는 큰 문제는 없지만, 문서의 가이드와 실제 구현이 약간 다르다는 점을 인지하세요.

파일 생성 확인: Report API 호출 후 실제로 **archive/reports** 디렉토리에 **파일이 생성되는지** 확인이 필요합니다. 코드에서 `reports_dir.mkdir(parents=True, exist_ok=True)`로 디렉토리를 생성하며 122, 그 경로는 기본적으로 `BASE/archive/reports`입니다 111. `BASE`는 `server_quali.py` 위치 기준 경로로, 프로젝트 루트일 경우 `<루트>/archive/reports`가 됩니다. Docker 이미지 내부에서도 `/app/archive/reports` 경로로 쓰이겠죠. 그리고 `StaticFiles` 마운트로 `/archive/reports/*` 요청시 해당 폴더 파일들을 서빙하므로, **응답 JSON의 path (`archive/reports/..`)는 UI 링크와 일치합니다**. (UI의 `absUrl(p)` 함수는 `location.origin`을 기준으로 주어진 경로 `p`를 풀어주도록 되어 있습니다 123.) 이를 통해, **파일이 존재하면 곧 바로 클릭하여 열람 가능하며, 파일이 없으면 링크 클릭 시 404 오류가** 나게 됩니다. 후자의 경우 UI 로그에 이미 error 메시지가 났을 것이므로 운영자가 알 수 있겠습니다.

JSON 파싱 및 캐싱 이슈: `/api/report`에서도 다양한 JSON을 읽는데, 혹시 **이전 단계에서 JSON이 잘못되었다면** 여기서 문제가 드러날 수 있습니다. 예를 들어 `selected_articles.json`이 빈 리스트이거나 invalid JSON이면, `candidates` loop를 돌다 `pass` 처리되고 `items`가 빈 상태로 남을 것입니다 106 124. 그러면 Markdown에는 “(수집된 기사 없음)”한 줄이 들어가게 됩니다 125. 이러한 경우 UI에도 “기사 없음”으로 표시된 Markdown 파일 링크가 나오므로 바로 알 수 있을 것입니다. 근본 원인은 `selection` 단계에서 `sync`가 제대로 안 된 것이므로, **report 결과가 비어있다면 selection 부분(특히 `selected_articles.json`)을 재점검**하면 됩니다.

UI 최신 빌드 반영: 이 단계에서 **캐시로 인한 이슈**라기보다는, **index.html 자체의 캐싱 문제**를 고려해야 합니다. 만약 사용자가 오래 열린 페이지에서 보고서 생성을 반복 호출하면, 항상 최신 JS 로직이 실행되는지 (또는 브라우저 캐시된 `index.html`이 오래된 코드일 위험)은 없을까 검토했습니다. 다행히 서버는 `index.html`을 서빙할 때 **Cache-Control: no-store, no-cache, must-revalidate** 헤더를 보내도록 설정했습니다 126. 이는 “항상 최신 HTML을 받아오라”는 지시이므로, **배포 후 UI 변경사항이 즉시 반영**됩니다. 또한 좌상단에 빌드 타임스탬프 배지가 있어 현재 UI 버전을 표시해 주므로, 운영자가 “최신 빌드 미반영” 문제를 쉽게 알아챌 수 있습니다 127. 혹시 업데이트가 적용되지 않는다면, **브라우저 캐시를 수동 삭제**하거나 `?v=` 쿼리 등을 붙여 재요청하면 될 것입니다 (그러나 현 설정상 자동으로 해결).

동시성 고려: `/api/report`는 서버 내 임계영역 리소스를 건드리지 않고 바로 파일을 쓰기에, 여러 사용자가 동시에 눌러도 큰 이슈는 없습니다. 다만 **동일 이름 파일 중복**의 경우, 기존 파일을 덮어씁니다. QualiJournal는 하루 한 번 발행 구조라 특별한 문제는 아니나, 혹시 **동일 날짜에 두 번 발행 시** `archive/reports` 내에 파일명이 충돌할 수 있습니다. 문서상 권고는 “중복발행시 파일명을 키워드_YYYYMMDD_HHMM 형태 등으로 조정해야”한다고 되어 있습니다 128. 현재 구현에서는 이를 처리하지 않으므로, **동일 날짜에 보고서 생성 여러 번 하면 마지막 것만 남게** 됩니다. 이를 완화하려면 `rollover_archive_if_needed()`와 유사한 기법으로, 기존 파일 있으면 리네임하거나 하는 처리가 필요합니다. 실제로 `/api/publish-keyword`에서는 발행 JSON/HTML이 중복될 경우 시간 스탬프를 붙여 `rollover`하는 코드를 구현했는데 129, `/api/report`에는 그런 처리가 없습니다. 이 점은 **알려진 제약**으로 문서에도 적혀 있으니, 운영 상 하루 한 번만 생성하는 것으로 관리하면 됩니다.

요약: 보고서 생성 단계는 API-UI 연동, 파일처리 모두 정상적으로 판단됩니다.

- [✓] **Markdown 파일 출력** - `archive/reports` 경로에 Markdown 파일이 생성되고 링크로 제공됨 ¹⁰².
- [✓] **UI 링크/다운로드 동작** - 응답 path를 받아 “열기” 링크로 표시, 클릭 시 Markdown 내용 다운로드 (인코딩 UTF-8) 가능 ¹¹⁷.
- [✓] **인증/권한** - 토큰 미포함시 401이나 UI disabled 처리지만, 자동 헤더 부착으로 문제 없음.
- [△] **중복 보고서 처리** - 동일 날짜 중복 생성시 기존 파일 덮어쓰 (필요시 수동 백업 or rollover 기능 추가 고려).

또한, **캐시 문제**는 index.html에 대한 no-cache 설정으로 해결되었고, **최신 빌드 미반영 우려는 적으나**, 혹시 **정적 파일 (e.g., JS)** 캐시 이슈가 있다면 빌드 배지를 통해 확인 가능합니다. (현재 JS는 index.html 내 인라인이 많아, HTML 캐시와 동일하게 관리됩니다.)

5. 발행 & 백업 (Publish & Backup) 단계 - 최종 발행 파이프라인과 백업 체계 점검

최종 발행: `/api/publish-keyword` (POST)은 명시적으로 키워드를 받아 **발행 작업**을 수행합니다 ¹³⁰. 이는 “특정 키워드에 대해 최종 뉴스 페이지를 생성”하는 API로, 내부에서 orchestrator의 `--publish-keyword <키워드>`를 실행합니다 ¹³¹ ¹³². 호출 시 **게이트 조건이 충족되어야 정상 완료**되며, 완료되면:

- **아카이브 디렉토리 구조:** `archive/` 아래에 해당 키워드 발행본의 JSON, Markdown, HTML이 생성됩니다 (예: `archive/2025-10-27_AI.json` 등) ¹³³ ¹³⁴. orchestrator publish 단계에서 Markdown/HTML 변환까지 담당하므로, 이 API에서는 orchestrator 호출만으로 모든 출력을 얻습니다.
- **롤오버 처리:** 동일 날짜 같은 키워드로 이미 발행본이 존재할 경우 `_rollover_archive_if_needed()`가 실행되어 기존 파일들을 `{base}_{HHMM}.{ext}` 형식으로 **이름 변경(백업)**합니다 ¹²⁹. 새로 생성될 파일들과 충돌을 막기 위한 조치로, 어떤 것이 새 것인지 명확히 구분하게 해줍니다.
`_rollover_archive_if_needed`의 반환값 `rollover` 리스트는 응답 JSON의 `"rolled_over": [...]` 필드로 제공됩니다 ¹³⁵ ¹³¹.
- **응답 구조:** orchestrator 실행 결과 (`ok`, `stdout`, `stderr`, `cmd`)를 기본으로, 추가로 `"created": [...]` 필드에 새로 생성된 파일들의 경로 리스트를 담아 반환합니다 ¹³². UI에서는 이 API를 **비동기 작업 또는 수동 모드** 둘 다 지원하게 만들었는데, 현재 inline 발행 버튼(키워드 특집 섹션의 “수집→승인Top20→발행”)은 `runKeyword()` 함수를 호출합니다 ¹³⁶. 이 함수는 **만약 백엔드가 비동기 작업(tasks) 기능을 지원하면** `/api/tasks/flow`를 사용하고, 아니면 동기식 fallback으로 `/api/flow/keyword`를 씁니다 ⁹². 여기서 `/api/flow/keyword`는 orchestrator 여러 단계를 순차 실행한 결과를 steps로 주며, **사실상 publish까지 포함**합니다 ¹³⁷. 즉, 동기 모드로 처리할 때는 `flow/keyword` 하나로 수집~발행을 모두 진행하고, 비동기 모드일 때는 tasks로 분리 수행합니다.

QualiJournal 최종 발행은 **HTML 생성과 외부 배포**가 연계될 수 있는데, 현재 구조에서는 Markdown/HTML이 archive에 쌓이는 것으로 끝납니다. cloudrun 등에서는 이 HTML을 외부 사이트와 연계하거나, 아니면 archive 폴더 자체를 static hosting하는 방식이 될 수 있습니다. 추가 고려사항이지만, **Admin 시스템 자체에서는 발행 결과물을 저장까지 책임지고, 그 이후(예: 메일 발송, 블로그 게시)는 운영 프로세스에 맡기는** 형태로 보입니다.

UI 측: UI에는 “키워드 특집” 섹션의 “수집→승인Top20→발행” 버튼이 존재하며, 편집자가 **하루 한 번 누르면** 전체 파이프라인이 돌아가도록 되어 있습니다 ¹³⁶. 이 버튼의 onclick은 `runKeyword()`로 연결되어 있고, 위에서 설명한 로직을 따릅니다. UI는 진행 과정을 **모달창** (`jobModal`)으로 표시하는데, tasks를 사용할 경우 실시간 SSE 로그 (`taskLog`)로, syncFallback(동기) 모드일 경우 즉시 steps 결과를 렌더링하는 차이가 있습니다 ¹³⁸ ¹³⁹.

만약 tasks 비동기 모드라면, UI `startFlow('keyword')` → `/api/tasks/flow` POST → `job_id` 받고 SSE 연결 이런 순서지만, 동기 **fallback 모드**(HAS_TASKS=false)에서는 `syncFallback('keyword')` 가 호출되어 곧바로 `/api/flow/keyword` 동작을 수행합니다 ¹⁴⁰. 이때 `syncFallback` 구현을 보면: `/api/flow/keyword` 를 호출한 뒤 steps를 합쳐 `renderSteps()` 와 `drawTimeline()` 으로 시각화합니다 ¹⁴¹ ¹⁴². 즉, “원클릭 발행” 버튼은 상황에 맞게 **orchestrator** 파이프라인을 전부 실행하며, UI에 각 단계 성공/실패를 로그로 보여 주고, 완료 후 `/api/status` 를 다시 불러 KPI 숫자를 업데이트합니다 ¹⁴².

검증 결과: 최종 발행 단계에서 중점적으로 본 것은:

- **Orchestrator publish 호출** - `/api/publish-keyword` 나 `/api/flow/keyword` 모두 `orchestrator.py --publish-keyword` 실행을 포함하고 있으며, **selected_articles.json**에 기반해 **Markdown/HTML 생성**까지 수행됨을 확인했습니다 ¹³¹ ⁸⁴. 오케스트레이터 스크립트 내부 구현을 직접 볼 수 없지만, 문서에서 “발행(Publish) 단계: 승인 ≥15 충족 시 정상 발행, 부족 시 `require_editor_approval` 임시 false로 응급발행 후 복구”라고 설명된 것으로 미루어 ⁹⁰, orchestrator는 **발행시 게이트를 검증**하고, 부족하면 (비상시) 설정 변경을 시도할 수 있습니다. 따라서 **발행 API 호출 결과를 반드시 확인**해야 합니다. 실패 시 `ok:false` 와 stderr에 오류 메시지가 담겨 UI에 표시될 것이며, “발행 실패” 토스트로 사용자에게도 알려집니다 ¹⁴³.

- **발행본 파일 확인** - orchestrator가 출력한 JSON/HTML/Markdown이 archive에 잘 저장되었는지 모니터링 합니다. `/api/publish-keyword` 응답의 `"created": [...]` 배열에 경로들이 나오므로 이를 확인 하면 됩니다 ¹³². 예시: `["archive/2025-10-27_AI.json", "archive/2025-10-27_AI.md", "archive/2025-10-27_AI.html"]`. UI에서는 동기모드일 경우 steps 리스트의 마지막에 `cmd: "... --publish-keyword"` 항목으로 stdout/stderr이 보이고, **생성된 파일 리스트는 응답 steps에는 안 담기지만** (steps는 orchestrator subprocess 결과만 포함), **별도 UI 표시를 할 수도 있습니다**. 현재 UI `renderSteps()` 는 각 step stdout을 그대로 로그에 찍는데, orchestrator publish 단계에서 stdout에 생성 파일명이 기록되도록 구현했을 가능성이 있습니다 (일반적으로 `make_daily_report`나 `orchestrator publish`는 “Saved as ...” 같은 로그를 출력하곤 하므로). 실제 `logger.info` 호출 부분을 보면 orchestrator subprocess 실행 전에 `logger.info("run orch: ... rc=...")` 찍고 stderr 있을 시 warning 남깁니다 ¹⁴. stdout 내용은 steps.stdout에 그대로 있겠쥬. 그러므로 UI 로그에서 publish 단계 stdout을 보면 어떤 파일 만들어졌는지 유추 가능할 것입니다.

- **자동 백업(rollover)** - 위에서 언급한 `rollover_archive_if_needed()`가 제대로 작동하는지 검토했습니다. 이는 동일 날짜+키워드 발행 JSON/HTML/MD가 이미 있을 때 실행되며, 중복본들을 `{시간} suffix`를 붙여 rename합니다 ¹²⁹. 이 과정에서 혹시 오류가 있으면 (파일 rename 불가 등) 예외는 없고, created 리스트에 이미 파일이 있었던 것들은 제외될 것입니다. 현재 `rollover` 결과는 응답의 `"rolled_over": [...]` 에 담겨 UI에는 따로 사용되지 않지만, 운영시 파일시스템을 통해 이전 발행본을 보존해두는 효과를 냅니다. 이 **백업(rollover) 기능은 제대로 구현되어 있으며**, 다만 Markdown 보고서 생성(`/api/report`)에서는 적용 되지 않음을 상기합니다. **정기 발행(API Publish)**에는 있으므로 큰 문제는 없습니다.

백업 상태 관리: QualiJournal Admin에는 **백업(Backup) 관련 API**도 존재합니다. `/api/backup/notify` (POST)와 `/api/backup/status` (GET)가 그것으로, Admin 시스템이 **외부 백업 프로세스와 연동**되는 부분입니다. 구조상, archive 폴더의 발행물들을 주기적으로 외부 저장소에 백업하거나 별도 DB에 저장할 수 있는데, 그 스크립트/프로세스가 끝나면 `/api/backup/notify` 에 결과(ok, timestamp, 파일 크기 등)를 보내 기록해두는 식입니다 ¹⁴⁴ ¹⁴⁵. 서버는 `_LAST_BACKUP` 전역변수에 이 정보를 저장하고 ¹⁴⁶, `/api/backup/status` 호출 시 JSON으로 리턴합니다 ¹⁴⁷. UI에서는 하단 우측에 “백업 상태” 배지를 표시하여, **마지막 백업 시간과 성공 여부를** 보여 줍니다 ¹⁴⁸ ¹⁴⁹. 예를 들어 24시간 이내 성공적 백업이면 녹색 “백업 OK · <시간>”으로, 지연되면 빨간 “백업 지연 · <시간>”으로 표시합니다 ¹⁵⁰. 이 기능은 **관리자 편의 기능**으로, 오류가 발생해도 핵심 파이프라인에는 영향이 없습니다. 실제 notify는 외부에서 호출해야 하고, status는 단순 조회이므로, **UI/서버 연결 이슈도 발견되지 않았습니다**.

UI 이벤트 바인딩 (발행 버튼): 마지막으로, UI에서 발행 관련 버튼들이 정상 작동하는지 살펴봤습니다. “키워드 특집”의 원클릭 발행 버튼은 inline onclick으로 잘 동작하고 ¹³⁶, 비동기 패널의 커뮤니티/데일리/키워드 실행 버튼들도 모두 inline onclick (`startFlow('daily')`) 등으로 연결되어 있습니다 ¹⁵¹. 여기엔 data-action 속성 등이 없고 직접 함수 호출방식이라 **바로 이벤트가 동작**합니다. (data-action이 쓰였던 “새로고침” 버튼들은 앞서 언급했듯 UI상 크게 문제되지 않는 부분이지만, 개선여지가 있습니다. `btnRefresh`는 수동 KPI 갱신용으로 보이나, 이미 `auto-refresh`와 `btnKpiRefresh`가 있어 중복일 수 있습니다. 현재 data-action 방식 이벤트핸들러가 코드상 없으므로 `btnRefresh` 클릭이 아무 동작 안 할 가능성이 있으니, 사소하나 이 부분은 추후 정리하면 좋겠습니다.)

결론: 발행 및 백업 단계는 **전체 파이프라인의 최종 출력과 안정성**에 관한 부분입니다. 코드와 구성을 종합해볼 때:

- [✓] **발행 명령 일관성** – orchestrator 기반 발행 (`--publish-keyword`)이 API로 노출되고 UI 버튼으로 실행 가능. 승인 기사 15개 이상 조건 준수 시 정상 완료 ⁹⁰.
- [✓] **파일 아카이빙** – JSON/HTML/MD 산출물이 archive 폴더에 저장되고, 중복시 자동 백업(rename) 처리되어 축적됨 ¹²⁹.
- [✓] **UI 피드백** – 발행 단계의 로그와 상태가 모달 및 토스트로 사용자에게 제공되어 투명성 확보 (성공시 “작업: flow-keyword · 상태: done”, 실패시 stderr 표시) ¹⁴² ¹⁴³.
- [✓] **백업 체계** – 시스템 내 백업 현황 모니터링 API/배지 구현으로, **주기적 백업 프로세스와 연계 가능**.
- [△] **사소한 UI 이벤트** – data-action 기반 버튼(새로고침 등) 몇 개는 이벤트 핸들러 부재로 작동 안 할 수 있음 (운영상 큰 문제 아님이나 개선 요망).

전체적으로 수집 → 선정 → 발행으로 이어지는 **파이프라인은 코드 논리의 앞뒤가 잘 맞고**, 필요한 파일 및 기능이 거의 갖춰져 있습니다. **주요 오류 원인**이 될 만한 부분은 크게 발견되지 않았으며, **Token 인증, CORS 등 연결 이슈**도 세심히 처리되어 있었습니다 ⁴⁶ ¹⁵². 발견된 몇 가지 **확인/개선 포인트**를 정리하면 다음과 같습니다:

- **Orchestrator.py 포함/동작 확인:** 배포 환경에 스크립트 누락되지 않았는지, 또 orchestrator의 `--collect-keyword`, `--approve-keyword-top`, `--publish-keyword` 옵션이 현재 코드 호출과 호환되는지 재검증 필요. (예: orchestrator가 `--approve-keyword-top 20` 옵션을 인식 못 하면 해당 단계가 효과 없을 수 있음. 코드에서 approve 순서를 `--approve-keyword-top 20 --approve-keyword <kw>` 로 주고 있는데, 혹 순서나 인자 문제 없는지 확인) ¹³⁷.
- **Ready 목록 조회 API 구현:** UI의 `loadReady()` 가 기대하는 `/api/items?state=ready` 엔드포인트가 없다면 추가 구현을 고려. 또는 간단히, `loadReady()` 함수 내부를 `/api/status` 데이터 활용으로 수정해도 됨. 현재까지는 selection JSON이 작아서 클라이언트 필터링해도 되지만, **전용 API가 더 정확**합니다.
- **data-action 이벤트 처리:** `btnRefresh`/`btnKpiRefresh` 버튼의 data-action (`status:refresh`, `status:kpi`)는 일반 클릭으로 아무 일도 안 할 가능성이 있습니다. 사용성 위해 **onclick 핸들러를 붙이거나, data-action 일괄 처리 루틴**(예: document.onclick 이벤트델리게이션) 구현을 추가해 주세요. (특히 `btnKpiRefresh`는 KPI 자동/수동 새로고침 개념으로 넣은 듯한데, 현재 `refresh()` 함수가 KPI도 갱신하므로 `btnRefresh`와 역할 중복입니다. UI 단순화를 고려할 수 있습니다.)
- **발행 중 게이트 유연성:** 현재 `require_editor_approval`이 True일 때 15개 미만이면 발행 API 자체가 실패하므로, 문서에서 언급한 “응급발행 모드”를 수동으로 처리하진 않았습니다. 필요하다면 `config.json`에서 `features.require_editor_approval`를 false로 토글하는 `/api/config/gate_required` API를 이용해 게이트 값을 낮춘 후 발행할 수 있습니다 ¹⁵³ ²⁷. 이 API는 UI에 슬라이더로 연결되어 게이트 값을 실시간 조정할 수 있게 되어 있습니다 ⁸⁷ ¹⁵⁴. 현재 기본 15로 설정되어 있고 UI KPI 패널에 슬라이더 (gateSlider)가 코멘트되어 있는 것으로 보여, **운영자는 UI에서 필요 시 임계값을 변경** 가능하니, 게이트로 인한 발행 중단은 없을 것으로 보입니다.

- **로그 확인 및 유지보수:** Admin UI에는 로그 조회 다운로드 기능도 있으므로 155 156 , deploy 환경에서 `logs/` 폴더 (orchestrator나 tasks 실행 로그 축적)를 주기적으로 모니터링/정리하면 좋겠습니다. 만약 **FastAPI 로거 설정**이 필요하다면 uvicorn 라인 등에서 `log_level` 조정 가능합니다 157 126 .

이상을 종합하면, **QualiJournal 관리자 시스템의 수집-발행 파이프라인은 전반적으로 일관되고 견고하게 구현**되었습니다. 일부 세부 기능 연결(예: Ready API)과 편의 개선 사항(data-action 이벤트, 중복처리 등)을 제외하면, **심각한 오류 없이 동작**할 것으로 판단됩니다. 현시점에서 치명적 버그는 발견되지 않았으며, 사소한 부분의 패치 여부만 검토하시면 됩니다.

마지막으로, 실제 운영에서 문제가 발생한다면 **다음 체크리스트**를 활용하세요:

- Orchestrator 실행 오류 시: 컨테이너에 `orchestrator.py` 있는지, `stdout/stderr` 내용(`logs/` 에서 or API 응답에서) 확인. 필요시 이미지를 재빌드하여 포함.
- 주요 API 401 에러 시: Admin 토큰 환경변수 설정 여부, UI 토큰 입력 여부 확인. **토큰 미설정(open mode)**이면 `authorize` 함수가 모든 요청을 통과시키므로, 이때 UI는 토큰 없음을 “미인증”으로만 표시하고 기능은 동작함을 인지 (UI에서 토큰 없을 때 `setActionsEnabled(false)` 로 일부 버튼을 막지만, 서버는 열려 있으므로 token을 입력하거나 UI 스크립트를 수정해 우회 가능).
- JSON 파일 관련 오류 시: `data/` 폴더 JSON들을 열어 구조/내용 확인. 필요한 경우 tools의 `repair_selection_files.py` 를 수동 실행해 구조 통일.
- UI 이상 동작 시: 브라우저 콘솔 에러와 네트워크 탭 확인. 특히 **fetch 실패(네트워크 실패)**나 JS 함수 미정의 에러 (safeWire 관련) 등이 없는지 확인. UI 빌드 배지 시각이 최신인지도 함께 점검.

이상으로 **전체 파이프라인을 단계별로 분석**하였으며, 현 시스템은 **설계 의도대로 수집→요약→선정→발행→백업 흐름이 이어지도록 구현**되어 있습니다. 각 단계에서의 **경로, 호출 관계, 에러 처리, UI 연계**는 대체로 정상적이므로, 발견된 미세한 부분만 개선하면 안정적인 운영이 가능할 것입니다. (**검토 완료**)

12659606365668385901004_2A고도화report.pdf

file:///file-Ko3WmavWUUXUmLMC1s6fso

3457891314171819202930313233343536373839404246474851

54676869707577787980818284889596102103104105106107108109111112113119122

124125126129130131132135137144145146147152155156server_quali.py

https://github.com/cdmacs1003-cyber/quali-journal/blob/49a0e32cd820715979aba34e9e890c8db9c30729/server_quali.py

1544455056575871727374878991929394979899101114115117118120123127136

138139140141142143148149150151154index.html

https://github.com/cdmacs1003-cyber/quali-journal/blob/49a0e32cd820715979aba34e9e890c8db9c30729/index.html

1623242526272841434952536176861001101161211331341531571011_3QualiJournal

관리자 시스템 최종 가이드북.pdf

file:///file-6Hu18vVrjQweSb5SZz6a6M

62641281004_3A작업계획report.pdf

file:///file-S3G4TOFPei3GeAJcXjN6B3